

1 Explain

1.1 explain 3D Gaussian Splatting in your own words

3D Gaussian Splatting 將 3D 空間中的點表示為高斯分佈。渲染過程的核心是 splatting（可以想像成把物體丟向螢幕）：將 3D 高斯分佈從其在 3D 空間中的位置，根據視角和方差投射到 2D 影像平面上，並進行加權疊加以生成最終影像。

訓練從稀疏點雲（由 SFM 生成）開始，逐步建立一組 3D 高斯分佈。通過 iterative optimization，不斷優化高斯的參數（包括位置、協方差、顏色和透明度）。渲染生成的影像與訓練視圖進行比較，並基於 L1 和 DSSIM loss 進行更新。

為了提升場景表示的細緻程度，採用 adaptive density control。對於細節不足的區域，模型會在高梯度區域生成新的高斯並沿著梯度方向移動；而對於過度重建的區域，則通過將高方差的大高斯分割成更小的高斯來捕捉細節。

1.2 Compare 3D Gaussian Splatting with NeRF (pros and cons)

NeRF 的 render 是基於 ray tracing，而 3D Gaussian Splatting 的 render 是基於 rasterization。

NeRF 的影像品質上限較高，特別是在細節和光影真實感方面，能表現出更自然的陰影與材質效果。但是 NeRF 的渲染速度相對較慢，且難以即時預覽場景，演算法較複雜。

3D Gaussian Splatting 的渲染速度遠快於 NeRF，這使其更適合即時視覺化和快速場景探索。然而，其渲染出的影像品質通常不如 NeRF，在細節表現和真實感上稍有欠缺。

1.3 Which part of 3D Gaussian Splatting is the most important you think? Why?

我認為 adaptive density control 是最重要的，因為它實現了高效性和高品質的平衡。

自適應密度控制直接影響了渲染結果的細節，透過動態地調整高斯的分佈密度，使得在邊緣或高頻細節區域的渲染更加細緻，避免過度模糊。此外，自適應的控制意味著可以顯著減少總的高斯數量，從而減少內存使用和計算需求，提升整體性能。

2 Describe the implementation details of your 3D Gaussian Spalting for the given dataset.

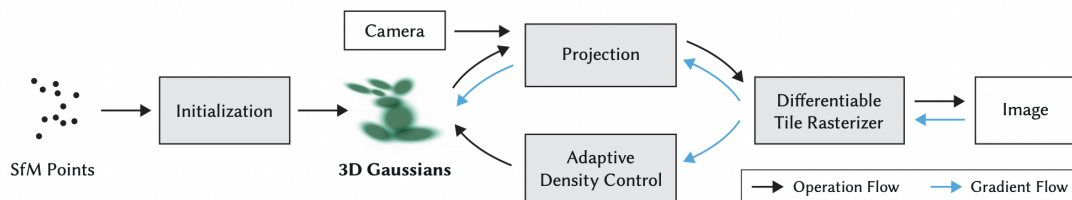


Fig. 2. Optimization starts with the sparse SfM point cloud and creates a set of 3D Gaussians. We then optimize and adaptively control the density of this set of Gaussians. During optimization we use our fast tile-based renderer, allowing competitive training times compared to SOTA fast radiance field methods. Once trained, our renderer allows real-time navigation for a wide variety of scenes.

Figure 1:

3D Gaussian Splatting 的實現從稀疏的 SFM 點雲初始化，構建一組包含位置、協方差矩陣、顏色和透明度等參數的 3D Gaussians。這些高斯分佈通過 differentiable tile-based rasterizer 投影到 2D 平面，並使用可微分的 alpha blending 方法計算重疊區域的合成顏色與透明度。

訓練過程中，對高斯參數進行迭代優化，使渲染圖像逼近真實視圖，loss function 考慮了 L1 以及 DSSIM。同時，自適應密度控制（adaptive density control）機制會調整高斯的數量和分佈：對於表示不足的區域，複製小高斯並沿梯度方向填補細節；對於過度表示的區域，將大高斯分裂為更小的高斯以提升細節表現。

Differentiable tile rasterizer 加速了渲染過程，實現了快速的場景排序與顯示，並支持高效的 α blending，適應高斯點數量的動態變化。

implementation details:

給定初始的 SFM points，首先將每個 3D 高斯分布的中心位置和協方差矩陣從世界坐標投影到相機坐標，再映射到 2D 像素空間，形成像素上的橢圓表示。接著計算每個像素受到所有高斯的影響，通過權重累加、顏色混合（如 α blending）合成最終影像，並用背景顏色補充未覆蓋區域。這過程是可微分的，允許通過損失函數優化高斯參數以逼近目標影像。我訓練時使用的參數如下（跟原 repo 相比，有把 iteration、percent_dense、lambda_dssim 調大以幫助學習）：

```
self.iterations = 250_000
self.position_lr_init = 0.00016
self.position_lr_final = 0.0000016
self.position_lr_delay_mult = 0.01
self.position_lr_max_steps = 30_000
self.feature_lr = 0.0025
self.opacity_lr = 0.025
self.scaling_lr = 0.005
self.rotation_lr = 0.001
self.exposure_lr_init = 0.01
```

```
self.exposure_lr_final = 0.001
self.exposure_lr_delay_steps = 0
self.exposure_lr_delay_mult = 0.0
self.percent_dense = 0.05
self.lambda_dssim = 0.6
self.densification_interval = 100
self.opacity_reset_interval = 3000
self.densify_from_iter = 500
self.densify_until_iter = 15_000
self.densify_grad_threshold = 0.0002
self.depth_l1_weight_init = 1.0
self.depth_l1_weight_final = 0.01
self.random_background = False
self.optimizer_type = "default"
```

3 Evaluate different settings with three metrics. Explain the meaning of these metrics

setting 1 和 setting 2 中，percent_dense, lambda_dssim, feature_lr, classification_interval 這幾個參數不同，其餘參數都和上一題提到的相同。

setting 1:

- percent_dense: 0.05
- lambda_dssim: 0.6
- feature_lr: 0.0025
- densification_interval: 100

setting 2:

- percent_dense 0.01
- lambda_dssim: 0.2
- feature_lr: 0.003
- densification_interval: 50

	PSNR	SSIM	LPIPS (vgg)	Number of 3D gaussians
setting 1	38.2866783	0.9805973	0.0339739	634079
setting 2	35.2775612	0.9680143	0.0669740	544222

可以看出 setting 1 更好地捕捉了場景的細節和結構特徵，從而得到的更好的影像，這可能是因為：

- 更高的 `percent_dense` 提供了足夠的高斯點密度，能夠更細緻地表現場景細節
- 更强的 DSSIM 約束保證了影像結構的相似性
- 適中的 Feature Learning Rate 提供了穩定的訓練過程
- 更長的 Densification Interval 讓模型有足夠時間學習分佈的穩定性
- 更多的高斯點捕捉更多細節

meaning of metrics:

- PSNR (Peak Signal-to-Noise Ratio): 透過計算重建影像和原始影像之間的均方誤差 (MSE)，並轉換為訊號雜訊比的形式來量化影像品質。PSNR 的數值越高，表示重建影像與原始影像越接近，品質越好。它常用於影像壓縮和去雜訊等任務。然而，由於 PSNR 僅關注像素級誤差，忽略人類視覺感知（如結構與對比），因此在感知品質評估中的準確性有限。
- SSIM (Structural Similarity Index): SSIM 衡量影像局部亮度、對比與結構的相似性，並基於這三者的加權結合來計算影像間的相似度。SSIM 的值範圍通常在 0 到 1 之間（1 表示完全相似），相比 PSNR 更能模擬人類視覺系統對影像結構與局部特徵的感知，特別適用於評估結構資訊的保留，如邊緣和紋理。
- LPIPS (Learned Perceptual Image Patch Similarity): LPIPS 通過計算兩張影像在預訓練深度學習模型（如 VGG）中間層特徵的加權差異來衡量它們的感知相似性。相比於基於像素的指標，LPIPS 能捕捉影像的高層次語義和感知特徵，其數值越低表示影像感知上越相似。

4 Train your 3D gaussians with random initializing points.

4.1 Describe how you initialize 3D gaussians.

用 `np.random.uniform` 隨機生成 13414 個點（和助教給的點雲數量相同），`x` 和 `y` 的範圍在 `[-1, 1]`，`z` 的範圍在 `[0, 2]`；每個點的顏色也是由 `np.random.uniform` 隨機生成的 RGB 值組成，範圍在 `[0, 1]`；用 `np.random.normal` 隨機生成每個點的法向量分量，再透過 `np.linalg.norm` 進行 `normalize`。

4.2 Compare the performance with that in previous question.

參數設定用的是上一題的 setting 1

	PSNR	SSIM	LPIPS (vgg)	Number of 3D gaussians
SFM initialization	38.2866783	0.9805973	0.0339739	634079
random initialization	19.7222366	0.7141145	0.3430109	2954971

可以看出隨機初始化 Gaussians 會導致影像品質較差，因為它缺乏場景的初始幾何結構，無法有效捕捉物體的輪廓和深度分佈，導致 optimization 過程緩慢且容易陷入局部最優。相比之下，SFM 點雲初始化提供了場景的基礎幾何資訊，讓模型能快速收斂並重建出更真實的影像。